Rust 语言程序设计考核方案

成绩构成：

- 课后作业：30%
- Rust 实践项目：70%
  - 项目设计报告：20%
  - 项目代码实现：40%
  - 项目答辩与演示：10%

---

1. 课后作业（30%）

课后作业分多个阶段布置，涵盖 Rust 的基础知识、高级特性和项目设计能力。

作业内容与时间安排

- 第 1-4 周：Rust 基础语法
  - 作业 1（5%）：变量、数据类型、控制流、函数、所有权和借用。
  - 作业 2（5%）：结构体、枚举、模式匹配、错误处理。
- 第 5-8 周：Rust 高级特性
  - 作业 3（5%）：生命周期、泛型、trait、闭包、迭代器。
  - 作业 4（5%）：并发编程（多线程、通道、互斥锁）。
- 第 9 周：项目设计与规划
  - 作业 5（10%）：撰写项目设计文档，包括项目目标、技术选型、模块划分和开发计划。

评分标准

- 代码正确性：50%
- 代码风格与规范：30%
- 文档与注释：20%

---

2. Rust 实践项目（70%）

实践项目是考核的核心部分，学生以 1 到 3 人小组形式完成一个完整的 Rust 项目。项目分为三个阶段：设计、实现和答辩。

2.1 项目设计报告（20%）

- 提交时间：第 10 周
- 内容要求：
  - 项目背景与目标：说明项目的实际意义和解决的问题。
  - 技术选型：解释为什么选择 Rust 以及使用的第三方库。
  - 系统架构：绘制系统架构图，描述模块划分和功能。
  - 开发计划：制定时间表，明确每个阶段的任务和分工。
- 提交形式：PDF 文档（5-10 页）。

- 评分标准：
  - 项目可行性：30%
  - 技术深度：30%
  - 文档清晰度：20%
  - 开发计划合理性：20%

2.2 项目代码实现（40%）
- 提交时间：第 15 周
- 内容要求：
  - 实现一个完整的 Rust 项目，代码需符合 Rust 的最佳实践。
  - 项目应包含以下内容：
  - 核心功能实现
  - 单元测试和集成测试
  - 代码文档（注释和 README 文件）
  - 使用 Cargo 管理依赖和构建
- 提交形式：
  - 代码仓库（GitHub/GitLab 等），包含完整的项目代码和文档。
  - 可运行的二进制文件或演示脚本。
- 评分标准：
  - 功能完整性：40%
  - 代码质量（可读性、模块化、错误处理）：30%
  - 测试覆盖率：20%
  - 文档完整性：10%

2.3 项目答辩与演示（10%）
- 时间安排：第 16 周
- 内容要求：
  - 每组进行 10-15 分钟的答辩，展示项目成果。
  - 答辩内容包括：
  - 项目背景与目标
  - 技术实现细节
  - 遇到的问题及解决方案
  - 项目演示（运行代码并展示功能）
  - 回答评委提问（5 分钟）。
- 评分标准：
  - 演示效果：40%
  - 技术深度：30%
  - 答辩表现：20%
  - 团队协作：10%

---

3. 项目选题建议

以下是一些适合 1 到 4 人完成的 Rust 项目选题，供学生参考：

1. 网络服务器：实现一个简单的 HTTP 服务器，支持静态文件服务和路由功能。
2. 并发数据处理系统：开发一个多线程日志处理器，支持实时数据分析和统计。
3. 区块链模拟：实现一个简单的区块链，包括区块生成、链式存储和共识算法。
4. 2D 游戏开发：使用 Amethyst 或 Bevy 引擎开发一个简单的 2D 游戏。
5. 嵌入式应用：在 Rust 中开发一个温度监控系统或机器人控制软件。
6. Web 后端服务：使用 Rocket 或 Actix 框架实现一个 RESTful API 服务。
7. 键值存储数据库：实现一个简单的键值存储系统，支持基本的 CRUD 操作。
8. 编程语言解释器：开发一个简单的解释器，支持算术表达式和变量赋值。
9. 网络安全工具：实现一个端口扫描器或简单的入侵检测系统。
10. 机器学习应用：使用 Rust 实现线性回归或决策树算法，并在数据集上进行测试。

4. 时间安排

周数	教学内容与任务
第 1-4 周	Rust 基础语法教学，布置作业 1-2
第 5-8 周	Rust 高级特性教学，布置作业 3-4
第 9 周	项目设计与规划教学，布置作业 5（项目设计文档）
第 10 周	学生提交项目设计报告，教师提供反馈
第 11-14 周	学生完成项目代码实现，教师定期检查进度并提供指导
第 15 周	学生提交项目代码实现
第 16 周	项目答辩与演示，教师评分

5. 评分细则

考核部分	占比	评分标准
课后作业	30%	代码正确性、风格与规范、文档与注释
项目设计报告	20%	可行性、技术深度、文档清晰度、开发计划合理性
项目代码实现	40%	功能完整性、代码质量、测试覆盖率、文档完整性
项目答辩与演示	10%	演示效果、技术深度、答辩表现、团队协作

6. 注意事项

- 每组需明确分工，并在设计文档中说明每个成员的贡献。
- 项目代码需通过代码相似性检查，杜绝抄袭行为。